

RAPPORT TECHNIQUE DU PROJET EN SYSTEMES D'EXPLOITATION

THEME :

Réimplémentation de la fonction Watch sous Linux

Membres du groupe 5 :

- ♦ SELIHE Emmanuel (**Chef**)
- ♦ TSAFAC TEUFAC Belvira Lins
- ♦ NDJOMO MELINGUI Emilienne Christelle
- ♦ BOULI ONANDA Grégory Kévin
- ♦ BELLA Gisèle Grace

Etudiants d'ingé 3 SRT

Sous la supervision de : M NGUIMBUS Emmanuel

2025-2026

Table des matières

INTRODUCTION	3
I. Architecture réelle et implémentation.....	4
II. Problèmes rencontrés	4
1. Gestion de l'affichage et contrôle du terminal (Séquences ANSI).....	4
2. Algorithme de différenciation visuelle (Mode Highlight)	5
3. Précision de la temporisation et gestion du type double	5
4. Interception des signaux et restitution du terminal	5
III. Guide d'installation et d'Utilisation	6
1. Environnement et prérequis du système	6
2. Protocole de déploiement et compilation (Makefile)	6
3. Manuel d'utilisation et syntaxe en ligne de commande	7
4. Contrôles interactifs au clavier	9
5. Gestion de l'environnement et codes de retour.....	9
IV. Tests réalisés.....	10
CONCLUSION.....	12

INTRODUCTION

Le présent rapport technique formalise la conception et la mise en œuvre de mywatch2, un clone optimisé de la commande système standard watch de Linux, entièrement développé en langage C par l'Équipe 5. L'objectif fondamental de ce travail d'ingénierie logicielle est de s'affranchir des abstractions de haut niveau de la bibliothèque standard pour interagir directement avec le noyau Linux par le biais d'appels système purs (syscalls). Face aux limitations de l'outil d'origine, l'architecture de mywatch2 introduit un analyseur d'arguments autonome conçu manuellement sans getopt (gestion des options -n, -b, -C, etc.), un moteur d'exécution asynchrone sécurisé basé sur le couplage de descripteurs de fichiers (pipe) et la duplication de processus (fork), ainsi qu'un module d'affichage dynamique capable d'intercepter les signaux d'entrée clavier à la volée. Ce document détaille les spécifications de bas niveau de chaque composant, explicite les choix algorithmiques retenus pour la manipulation des flux système, et justifie les structures de données implémentées pour garantir la robustesse et la modularité de notre application.

I. Architecture réelle et implémentation

L'architecture logicielle de mywatch2 est structurée de manière modulaire autour de types de données stricts et d'abstractions de bas niveau afin d'isoler chaque responsabilité applicative. Le pilotage des configurations globales est centralisé dans le fichier d'en-tête `args.h` à travers la structure pivot `t_args`, qui encapsule l'ensemble des indicateurs de fonctionnalités (tels que `interval` pour la périodicité, `highlight` pour le surlignage, `beep` pour l'alerte sonore, ou encore `no_color` pour le filtrage). Le traitement de la ligne de commande est dévolu à la fonction `parse_args` du module `args.c`, qui inspecte manuellement le vecteur `argv` sans dépendance à `getopt`, extrait les variables d'environnement comme `WATCH_INTERVAL` et valide la présence d'une commande utilisateur. Le cœur fonctionnel du système est opéré par `executor.c` via l'implémentation de la routine `execute_cmd`. Ce module assure l'isolation des tâches par l'exécution synchrone d'appels système purs : il instancie un canal de communication IPC via `syscall(22, tuyau)`, segmente l'application par un clonage de processus avec `syscall(57) (fork)`, puis exploite `syscall(33) (dup2)` dans le processus fils pour dérouter les descripteurs de flux standards (`stdout/stderr`) vers l'entrée du tube avant d'invoquer la commande par un mécanisme d'exécution directe ou de recherche de chemin. Le processus père récupère dynamiquement ce flux textuel par blocs de 4096 octets via `syscall(0) (read)` et réalloue la mémoire dynamiquement jusqu'à épuisement du canal, tout en prévenant l'apparition de processus zombies grâce à une attente bloquante régie par `waitpid (syscall(61))`. Enfin, la restitution visuelle et le traitement des chaînes sont coordonnés par `display.c`, qui orchestre l'effacement sélectif de la console par séquences d'échappement ANSI et délègue le nettoyage des attributs de style au module `color.c` lorsque l'option `-C (no_color)` est activée, garantissant une restitution de texte brute et sécurisée à l'écran.

II. Problèmes rencontrés

1. Gestion de l'affichage et contrôle du terminal (Séquences ANSI)

Le problème : Lors des premières versions de l'outil, l'utilisation de fonctions de nettoyage classiques (comme un enchaînement de sauts de ligne ou la commande système `clear`) provoquait un clignotement permanent de l'écran (`flickering`) et polluait l'historique du terminal de l'utilisateur en faisant défiler le texte verticalement.

L'analyse : La commande d'origine `watch` n'écrit pas dans le flux standard de manière linéaire, elle fige l'affichage dans un espace tampon isolé du terminal global.

La solution : L'implémentation des séquences d'échappement ANSI spécifiques dans le module `display`. Le passage à l'écran alternatif (`\033[?1049h`), le masquage du curseur (`\033[?25l`) et le repositionnement systématique du curseur en haut à gauche (`\033[H`) ont permis d'obtenir un rendu plein écran stable, fluide et parfaitement isolé.

2. Algorithme de différenciation visuelle (Mode Highlight)

Le problème : La mise en surbrillance des changements (option -d) a posé des difficultés de précision. Comparer de simples chaînes de caractères de tailles différentes ou contenant des caractères spéciaux (retours à la ligne \n, tabulations) provoquait des décalages d'affichage ou des faux positifs (tout le reste du texte passait en surbrillance dès qu'un caractère était ajouté au début).

L'analyse : Une comparaison brute globale est insuffisante. Il est nécessaire d'analyser le flux de données de manière séquentielle, octet par octet, tout en gérant les limites de taille de la mémoire tampon précédente (\$t-1\$) et actuelle (\$t\$).

La solution : Développement d'une boucle itérative stricte dans la fonction `display_output`. Le programme compare l'index `$i$` des deux tampons et applique la séquence d'inversion vidéo `\033[7m` de manière chirurgicale, uniquement sur l'octet modifié, avant de réinitialiser immédiatement le style avec `\033[0m`.

3. Précision de la temporisation et gestion du type double

Problème : La commande standard doit pouvoir accepter des intervalles très courts ou décimaux (par exemple 0.5s). L'utilisation initiale de la fonction standard `sleep()` de la bibliothèque `<unistd.h>` s'est révélée bloquante, car elle n'accepte que des valeurs entières (en secondes).

L'analyse : Pour obtenir une précision de l'ordre de la milliseconde, les structures de données de l'analyseur d'arguments (`args.h`) devaient abandonner le type `int` au profit du type `double`, nécessitant des fonctions de temporisation système plus avancées.

La solution : Remplacement de `sleep()` par la fonction `usleep()` (qui prend des microsecondes) ou par la structure `nanosleep()`. Une conversion mathématique rigoureuse a été intégrée pour traduire l'argument `double` en microsecondes, permettant un respect strict du rythme de rafraîchissement demandé par l'utilisateur.

4. Interception des signaux et restitution du terminal

Le problème : Lorsque l'utilisateur quittait brusquement le programme en cours d'exécution avec la combinaison de touches `Ctrl+C`, le terminal de l'hôte restait "cassé" : le curseur restait invisible, les couleurs étaient inversées et l'affichage demeurait bloqué sur l'écran alternatif.

L'analyse : Par défaut, le signal `SIGINT` tue le processus instantanément, empêchant le programme d'atteindre la fin de la fonction `main` et d'exécuter le nettoyage.

La solution : Mise en place d'un gestionnaire de signaux (Signal Handler) via la fonction `signal(SIGINT, ...)`. Ce déroutement permet d'intercepter la demande de fermeture pour exécuter explicitement la fonction `display_cleanup()` avant de couper définitivement le programme, garantissant la restitution d'un terminal propre à l'utilisateur.

III. Guide d'installation et d'Utilisation

Cette section fait office de manuel technique de référence (README). Elle décrit le protocole rigoureux mis en place pour le déploiement opérationnel, la compilation sécurisée du code source en langage C et l'exploitation des différentes options en ligne de commande de l'utilitaire mywatch2.

1. Environnement et prérequis du système

Pour garantir le bon fonctionnement de l'application et l'exécution des appels système directs (syscall), l'environnement d'accueil doit valider les exigences suivantes :

- **Système d'exploitation** : Distribution Linux native (Ubuntu, Debian, Kali Linux) ou couche de compatibilité WSL (Windows Subsystem for Linux).
- **Chaîne de compilation** : Compilateur GCC (GNU Compiler Collection) embarqué dans le méta-paquetage de développement d'outils de base.

Bash

```
sudo apt update && sudo apt install build-essential -y
```

2. Protocole de déploiement et compilation (Makefile)

La gestion de la compilation du projet a été automatisée à l'aide d'un outil de build (Makefile). Le processus applique strictement les drapeaux de sécurité et d'optimisation ISO C99 suivants : -Wall (alertes de syntaxe), -Wextra (alertes de conception) et -Werror (traitement de la moindre alerte comme une erreur bloquante).

1. Clonage du dépôt et positionnement

Bash

```
git clone https://github.com/OpenSecureFoundation/mywatch.git
```

```
cd mywatch
```

2. Les commandes d'automatisation (Cibles du Makefile)

Le cycle de vie de la construction du binaire s'articule autour de quatre commandes clés :

- **Compilation standard (make)** : Analyse les dépendances des modules (main.c, args.c, executor.c, display.c, utils.c, shell.c), génère les fichiers objets .o et assemble le binaire final nommé mywatch2.

Bash

```
make
```

- **Nettoyage des fichiers intermédiaires (make clean)** : Purge le répertoire de travail en supprimant l'intégralité des fichiers objets compilés pour libérer l'espace.

Bash

make clean

- **Recompilation intégrale (make re) :** Force une reconstruction propre et totale du projet depuis zéro en enchaînant un nettoyage puis une nouvelle compilation.

Bash

make re

3. Manuel d'utilisation et syntaxe en ligne de commande

1. Structure de la syntaxe générale

L'exécutable s'invoque de manière standard via le terminal en respectant le formalisme suivant :

Bash

`./mywatch2 [OPTIONS] COMMANDE [ARGUMENTS...]`

2. Registre exhaustif des options de configuration (Flags)

Option / Flag	Rôle Technique d'Analyse	Exemple d'Application Concrète
-n <sec>	Fixe la période de la boucle d'attente (Défaut : 2.0s).	<code>./mywatch2 -n 1 date</code>
-d	Active le surlignage visuel des modifications (Vidéo inversée).	<code>./mywatch2 -d ls</code>
-t	Masque l'en-tête (Informations système, heure et intervalle).	<code>./mywatch2 -t date</code>
-e	Interrompt la surveillance si la commande renvoie un statut <code>\$\neq 0\$</code> .	<code>./mywatch2 -e make</code>

Option / Flag	Rôle Technique d'Analyse	Exemple d'Application Concrète
-g	Quitte instantanément le programme si l'output du cycle \$T\$ diverge de \$T-1\$.	<code>./mywatch2 -g date</code>
-b	Déclenche un signal sonore d'alerte matériel (a) sur code d'erreur.	<code>./mywatch2 -b make</code>
-p	Force l'utilisation du mode d'attente haute précision (<code>precise_sleep</code>).	<code>./mywatch2 -p -n 1 date</code>
-q <n>	Ferme le programme si l'affichage reste statique durant \$n\$ cycles.	<code>./mywatch2 -q 3 echo bonjour</code>
-w	Aligne l'affichage en tronquant les lignes dépassant la largeur du terminal.	<code>./mywatch2 -w cat /etc/passwd</code>
-c	Force l'interprétation et le rendu des couleurs ANSI de la sous-commande.	<code>./mywatch2 -c ls --color=always</code>
-C	Filtre et neutralise toutes les séquences de couleurs ANSI de la sortie.	<code>./mywatch2 -C ls --color=always</code>
-f	Désactive le mode plein écran et passe en mode défilement (<i>scroll</i>).	<code>./mywatch2 -f date</code>
-x	Exécute la commande de manière isolée sans invoquer le sous-shell <code>/bin/sh</code> .	<code>./mywatch2 -x date</code>

Option / Flag	Rôle Technique d'Analyse	Exemple d'Application Concrète
--help / -h	Interrompt l'exécution pour afficher l'aide interactive détaillée.	<code>./mywatch2 --help</code>
-v	Affiche la version courante de l'application (Équipe 2025-2026).	<code>./mywatch2 -v</code>

4. Contrôles interactifs au clavier

Grâce à l'intégration de l'appel système `select` et de l'écoute asynchrone codée dans le module `utils.c`, l'utilisateur dispose de commandes interactives en temps réel directement depuis le terminal pendant le fonctionnement de la surveillance :

- **Touche q ou Q (Quit)** : Déclenche la sortie de la boucle infinie, invoque la fonction de nettoyage `display_cleanup` pour restituer l'état initial du terminal hôte et ferme l'application proprement.
- **Touche Espace (Force Refresh)** : Court-circuite la minuterie temporelle ou l'attente du cycle en cours pour forcer l'exécution immédiate de la commande ciblée et rafraîchir l'écran à l'instant T.
- **Touche s ou S (Snapshot)** : Capture l'intégralité de la mémoire tampon textuelle affichée à l'écran et génère instantanément une sauvegarde sous la forme d'un fichier d'export au format texte brut (`.txt`).

5. Gestion de l'environnement et codes de retour

1. Persistance par variable d'environnement

Le programme est conçu pour être à l'écoute de l'environnement système de l'utilisateur. Si une variable nommée `WATCH_INTERVAL` est exportée dans le shell, `mywatch2` l'adopte automatiquement comme valeur temporelle par défaut à la place des 2.0 secondes initiales :

Bash

```
export WATCH_INTERVAL=5
```

```
./mywatch2 date # S'exécutera automatiquement toutes les 5 secondes
```

2. Table de correspondance des codes de sortie du programme (`exit_status`)

À l'arrêt de l'application, l'environnement shell reçoit un indicateur numérique précis reflétant la nature de la terminaison :

Code de sortie	Signification de l'état système
0	Succès nominal : Interruption demandée par l'utilisateur (Ctrl+C ou 'q') sans anomalie.
1	Erreur d'initialisation : Argument manquant ou option invalide passée à la fonction <code>parse_args</code> .
2	Erreur d'infrastructure : Échec d'allocation mémoire, rupture d'un canal IPC (pipe) ou commande introuvable.

IV. Tests réalisés

Cette section présente les tests effectués pour valider le bon fonctionnement du programme `mywatch2`.

Test 1 — Exécution normale avec rafraîchissement

Commande lancée : `./mywatch2 -n 2 -d ls -l`

Résultat observé : le programme affiche correctement l'en-tête sur la première ligne :

```
Every 2.0s: ls -l      Mon Jun 15 10:04:08 2026
total 136
-rw-rw-r-- 1 root root 1078 Apr 25 05:20 LICENSE
-rw-rw-r-- 1 root root 297 Jun 13 17:41 Makefile
-rw-rw-r-- 1 root root 88 Apr 25 05:20 README.md
-rw-rw-r-- 1 root root 3021 Jun 13 14:56 args.c
-rw-rw-r-- 1 root root 415 Jun 13 15:06 args.h
-rw-rw-r-- 1 root root 927 Jun 13 14:10 color.c
-rw-rw-r-- 1 root root 87 Jun 13 14:07 color.h
-rw-rw-r-- 1 root root 1798 Jun 13 14:08 display.c
-rw-rw-r-- 1 root root 204 May 17 23:16 display.h
-rw-rw-r-- 1 root root 605 May 19 05:30 error.c
-rw-rw-r-- 1 root root 306 May 19 05:26 error.h
-rw-rw-r-- 1 root root 3288 Jun 13 15:03 executor.c
-rw-rw-r-- 1 root root 117 Jun 13 15:05 executor.h
-rw-rw-r-- 1 root root 836 May 21 21:05 help.c
-rw-rw-r-- 1 root root 68 May 21 21:05 help.h
-rw-rw-r-- 1 root root 1310 Jun 13 07:04 keyboard.c
-rw-rw-r-- 1 root root 172 Jun 13 06:54 keyboard.h
-rw-rw-r-- 1 root root 3393 Jun 13 17:47 main.c
-rwxrwxr-x 1 root root 22536 Jun 13 17:48 mywatch2
-rw-rw-r-- 1 root root 773 Jun 13 17:43 screenshot.c
-rw-rw-r-- 1 root root 100 Jun 13 17:37 screenshot.h
-rw-r--r-- 1 root root 29 Jun 13 17:48 screenshot_2026-06-13_17-48-24.txt
-rw-r--r-- 1 root root 29 Jun 13 17:48 screenshot_2026-06-13_17-48-27.txt
-rw-rw-r-- 1 root root 718 Jun 13 15:02 shell.c
-rw-rw-r-- 1 root root 81 Jun 13 15:02 shell.h
-rw-rw-r-- 1 root root 668 May 17 23:15 utils.c
-rw-rw-r-- 1 root root 289 May 17 23:14 utils.h
-rw-rw-r-- 1 root root 1275 Jun 13 14:08 wrap.c
-rw-rw-r-- 1 root root 166 Jun 13 14:08 wrap.h
Au revoir !
(root@kali) ~/projets/mywatch
```

On y retrouve l'intervalle de rafraîchissement (2 secondes), la commande exécutée (ls -l) ainsi que la date et l'heure courante, mise à jour à chaque cycle. Le contenu du répertoire est ensuite listé avec les permissions, la taille et la date de modification de chaque fichier. L'option -d est active : toute différence entre deux affichages consécutifs sera surlignée automatiquement. En bas de l'écran, le message Au revoir ! confirme que l'utilisateur a quitté proprement le programme en appuyant sur la touche q.

Test 2 — Gestion des erreurs (option invalide)

Commande lancée :

```
(root@kali) ~/projets/mywatch
$ ./mywatch2 -z
ERREUR : option inconnue

(root@kali) ~/projets/mywatch
$ ./mywatch2
Usage: ./mywatch2 [-n sec] [-d] [-t] [-e] commande

(root@kali) ~/projets/mywatch
$ q
```

Résultat observé : le programme détecte immédiatement que l'option -z est inconnue et affiche: **ERREUR : option inconnue**

Usage: ./mywatch2 [-n sec] [-d] [-t] [-e] commande

Le message d'usage est affiché pour guider l'utilisateur, puis le programme se termine sans entrer dans la boucle principale. La gestion d'erreur fonctionne correctement.

Ces deux tests valident les comportements essentiels du programme : l'affichage périodique avec en-tête horodaté, la sortie propre via q, et le rejet des options invalides avec message d'erreur explicite

CONCLUSION

En définitive, le développement de mywatch2 concrétise l'application des concepts fondamentaux de la théorie des systèmes d'exploitation à travers une implémentation logicielle rigoureuse en langage C. En relevant le défi de s'affranchir des fonctions d'abstraction standard pour orchestrer directement des appels système de bas niveau — tels que `syscall(57)` pour le clonage de processus, `syscall(22)` pour la communication par tube et `syscall(0)` pour la capture asynchrone des flux d'entrée —, l'Équipe 5 a conçu une alternative robuste et hautement modulaire. La structuration claire des données au sein de `t_args`, combinée à une séparation stricte des rôles entre le parseur manuel, le moteur d'exécution et les filtres de nettoyage de l'affichage, assure une excellente stabilité de l'outil lors de la surveillance en temps réel. Au-delà de la validation technique du cahier des charges et de l'optimisation des performances de bas niveau, ce projet d'ingénierie SRT aura consolidé notre maîtrise de la gestion des ressources du noyau Linux et renforcé nos compétences en développement système collaboratif.